```
$newCars = [];
$newKeys = array_keys($car);
shuffle($newKeys);

foreach ($newKeys as $k) {
    $newCars[$k] = $car[$k];
}

print_r($newCars);

Output:
Array
(
    [variant] => GTI 8v
    [make] => VW
    [colour] => Oak Green
    [model] => Golf
)
```

The output will vary every time, of course, but that's the point! An astute person might perhaps wonder if there was a way to leverage `uasort()` for a similar purpose, perhaps something like this:

```
uasort($arr, function () {
    return rand(-1, 1); // returns a random int -1, 0, or 1
})
```

The answer is no. `uasort()` implements the Quicksort algorithm; you will find (especially with large arrays) that the "randomized" keys will tend to stick near to their original positions (because the algorithm partitions the original array into sub-arrays first).

A final word of warning about `shuffle()`: it is not cryptographically secure. If you need the new order to be unguessable, you will need to create your own function based on `random_int()`, `random_bytes`, or the `Random\Randomizer` API. See Chapter 7, "Numbers," for more detail.